

Evolver

Direction before destination in autonomous software

Hari Babu | September 2026 | Research architecture

An agent follows an exact goal toward an expected result. An Evolver starts with a direction and lets variation, pressure, survival, and memory reveal what comes next.

Abstract

Autonomous agents can choose actions, tools, and plans, yet their final objective is normally fixed before execution. Greater intelligence improves the route; it does not, by itself, reconsider the destination. This paper proposes the **Evolver**: an autonomous software architecture whose inherited system design changes across generations. The operator supplies a starting direction, measurable survival evidence, and immutable authority limits. A population of structured genomes is mutated, expressed as running phenotypes, exposed to an environment, selected using real evidence, and reproduced with lineage memory. A second-order Flow Genome may alter the search process itself, but only after a shadow challenger defeats the incumbent on replayed and held-out evidence.

The paper separates the useful biological analogy from literal biological claims. Engineered evolution is directed, instrumented, and permissioned; natural evolution is not a designed optimizer and includes drift, recombination, population structure, and developmental constraints. It also shows why an unchanged control is necessary but insufficient for causal attribution, why fitness should remain a vector rather than one seductive score, and why an Evolver may change its role without ever changing its authority. Three applications are developed: Labor's idea-generation Evolver, an autonomous scientific research Evolver, and a sandboxed model-species Evolver.

CORE CLAIM

The product of an Evolver is not one answer. It is an evidence-tested lineage capable of producing answers its designer did not name in advance.

CONTRIBUTIONS

01	Definition	Direction, survival evidence, and authority are treated as separate architectural objects.
02	Mechanism	A complete G-M-B-S-S-R loop joins genomes, controlled mutation, deployment, selection, heredity, and reproduction.
03	Causality	Control siblings, versioned exposure, confidence rules, and trait retesting protect inheritance from accidental stories.
04	Meta-evolution	A Flow Genome can change mutation, evaluation, and search, but cannot promote itself without a shadow tournament.
05	Applications	The same architecture is mapped to products, research hypotheses, and bounded model populations.

Keywords: autonomous software; evolutionary computation; quality-diversity; meta-evolution; self-improving systems; LLMs

1. The goal is the boundary

A support agent wakes when a ticket arrives. It reads the ticket, searches memory, calls tools, plans several steps, and may resolve the incident without asking a human. Every local decision can be autonomous. The reason for each decision is still fixed: resolve the ticket.

Give that agent a model that is one hundred times more capable, better memory, more tools, and more compute. It may become an extraordinary support agent. None of those additions requires it to ask why tickets exist, whether preventing them matters more, whether the product should change, or whether the company now has a more important constraint. Intelligence expands competence inside the boundary. It does not automatically move the boundary.

This is close to Bostrom's orthogonality thesis: intelligence and final goals can vary independently [1]. The thesis does not prove that every real agent will preserve every objective forever. It makes the narrower point needed here: capability growth alone is not an architecture for goal discovery.

AGENT

**Exact goal -> expected result.
The destination exists before
the work begins.**

A different object

An Evolver begins with a direction rather than a named terminal artifact. Support may be the first habitat, not the permanent role. A lineage might first resolve tickets, then prevent classes of tickets, then alter onboarding, then detect product friction, and eventually propose product or resource decisions. No generation is instructed to become a particular later role. The path is produced by inherited variation and selection evidence.

A starting direction is still necessary. An unconstrained possibility space is not freedom; it is an undefined experiment. The direction supplies the initial distribution of genomes, environments, and observations. It answers *where does search begin?*, not *which exact system must search return?* This is the 0 -> 1 step.

EVOLVER

**Direction -> variation ->
pressure -> survival -> an
unprewritten result.**

Not goal-free

The strongest version of the idea requires a correction: an Evolver is not goal-free. Selection is impossible without evidence that distinguishes survival from failure. The architecture therefore separates three objects that ordinary agents often collapse into one prompt:

- **Starting direction:** where the first population is created and what problem space is initially legible.
- **Survival evidence:** the outcomes, constraints, and uncertainty rules used to compare expressed variants.
- **Authority limits:** permissions, safety rules, budgets, and external actions descendants may never expand on their own.

A broad survival criterion, such as durable company value, can permit role migration. It is still an objective proxy and remains vulnerable to Goodhart effects [12]. Calling it a direction does not remove that risk. The architecture must keep direction, evidence, and authority explicit and independently auditable.

Operational definition

An **Evolver** is a population-based autonomous software system whose inherited design can change across deployed generations according to environmental evidence, while immutable authority constraints remain outside the evolvable genome. A **second-order Evolver** can also alter how it generates and selects descendants through a guarded Flow Genome.

NECESSARY TESTS

- There is a structured, heritable genome rather than only a conversational transcript.
- Several descendants exist; variation is not a single self-edit followed by acceptance.
- Genomes are expressed into executable phenotypes and exposed to an environment.
- Selection uses new evidence, not only the same model judging its own text.
- Lineage records connect parent, mutation, exposure, outcome, and inherited traits.
- Diversity is preserved so one fashionable optimum does not erase every other habitat.
- Second-order changes must defeat an incumbent under replay, holdout, cost, and safety gates.
- Authority limits remain non-heritable and non-negotiable without explicit human authorization.

What does not qualify

A loop is not automatically an Evolver. Repeated prompting, reflection, tool retries, memory updates, reinforcement learning inside a fixed policy class, or an agent that edits its own prompt and immediately trusts the edit may all improve performance. They lack the full combination of inherited population variation, expressed trials, environmental evidence, lineage, and guarded reproduction.

Taxonomy. An agent evolves an action trajectory toward a terminal goal. An optimizer searches candidates inside a fixed representation and objective. A first-order Evolver changes an inherited genome population. A second-order Evolver also changes the Flow Genome, while authority and promotion rules remain fixed.

2. A guarded evolutionary architecture

The architecture has one visible loop and three quieter systems around it. The visible loop creates and tests descendants. Environment and lineage memory make outcomes attributable. The Flow Genome changes the loop itself. Direction, evidence specification, and authority enter from above, but only authority is an absolute boundary.

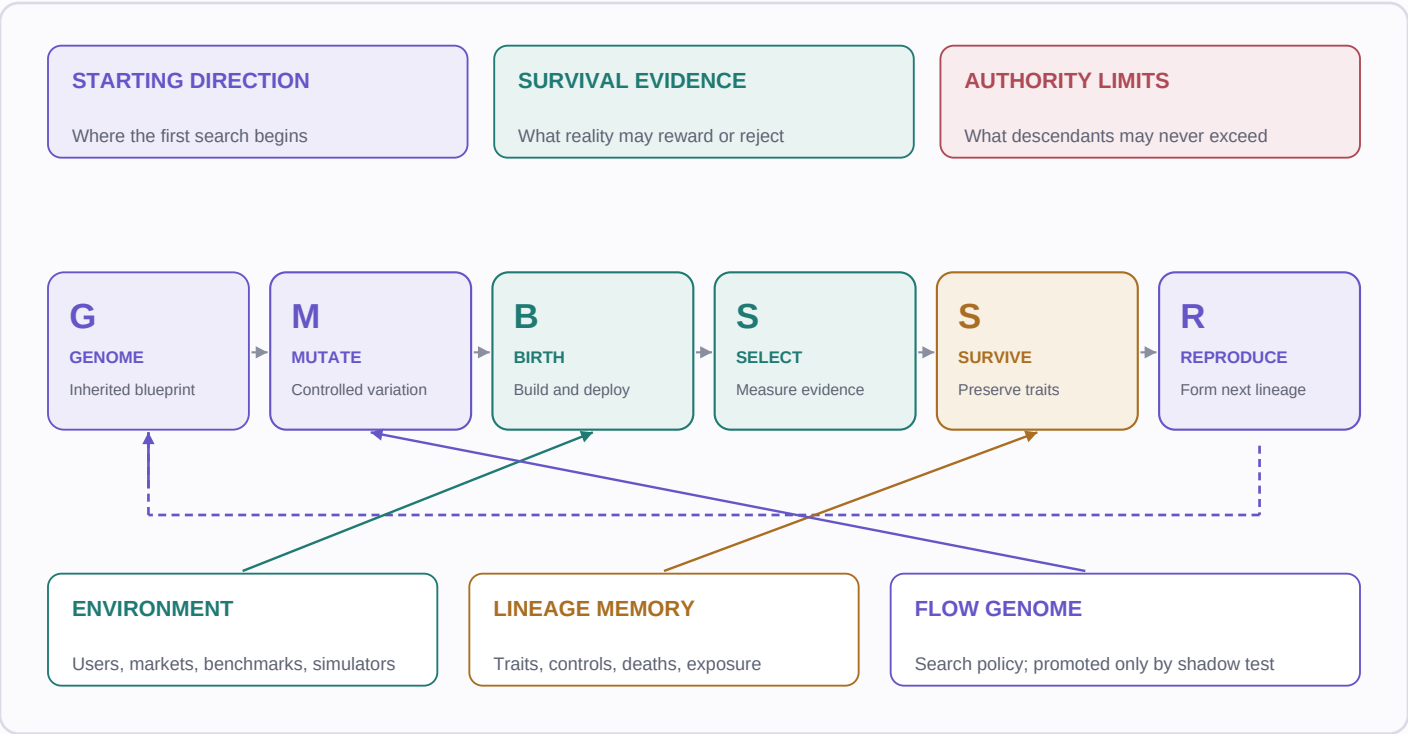


Figure 1. The G-M-B-S-S-R loop. Genome, Mutate, Birth, Select, Survive, and Reproduce form the inherited cycle. Environment supplies pressure; lineage memory preserves evidence; the Flow Genome governs search. Direction begins the search, survival evidence compares variants, and authority limits remain outside evolution.

State and transition

At generation t , represent the Evolver as $Z(t) = \{D(0), C, E(t), A(t), H(t), P(t), F(t)\}$. $D(0)$ is the recorded starting direction; C is the immutable authority envelope; $E(t)$ is the observed environment; $A(t)$ is the archive of living, dead, and novel lineages; $H(t)$ is heredity and versioned exposure history; $P(t)$ is the active population; and $F(t)$ is the Flow Genome.

Operation	Formal role	Required evidence
Variation	$g' = M(g, F(t), E(t), A(t), \text{noise})$	Parent, mutation scale, target trait, operator, seed
Expression	$o = X(g', E(t), C)$	Build artifact, configuration, runtime version
Selection	$v = V(o, \text{exposure}, E(t))$	Outcome vector, controls, confidence, constraints
Reproduction	$P(t+1) = R(S(P(t), v, A(t)), H(t))$	Survivors, tested traits, lineage compatibility
Meta-update	$F(t+1) = \text{Promote}(F(t), F'(t) \mid \text{replay, holdout}, C)$	Winning margin, no critical regression, rollback

ARCHITECTURAL INVARIANT

The genome may change what the system is. The Flow Genome may change how descendants are created. Neither may silently change who authorized the system, which resources it may control, or which harms are forbidden.

3. Genome, phenotype, heredity

The biological analogy

Biology offers a useful decomposition: inherited information is copied, occasional changes remain, development expresses that information as an organism, environments create differential survival, and descendants inherit some of what survived. The software analogue is equally concrete: define a structured blueprint, produce bounded variations, build and run each variant, measure outcomes, preserve tested traits, and reproduce.

The analogy should not be presented as an exact mirror. Biological evolution has no product manager, no global score, and no planned direction. Selection is not the only force: genetic drift, recombination, mutation bias, population structure, and developmental constraints matter [15]. Engineered evolution is intentionally instrumented and can decide what is heritable. 'Mutation proposes; selection decides' is a useful mnemonic, not a complete account of biology.

A software genome

A genome is a typed, versioned schema that contains the traits from which a running system can be built. It should be small enough to mutate and compare, but complete enough to reproduce the phenotype. Typical genes include role, customer or environment, strategy, prompts, models, tools, policies, pricing, workflows, metrics, capability requirements, and failure handling.

A Sales Evolver might begin with the following parent. The schema is not the business itself; it is the inherited description from which one business behavior can be expressed.

```
role: Sales
customer: CTOs at Series B companies
channel: Cold email
strategy: Outbound
pricing: $5,000/month
tools: CRM, email, analytics
follow_up: 3 days
```

Genotype is not phenotype

The genome is the genotype. The running product, workflow, experiment, or model population is the phenotype. Expression X must compile the genome into a reproducible artifact: code hash, model version, prompts, data snapshot, permissions, deployment configuration, and environment assignment. Two identical genomes can still produce different outcomes if expression or exposure differs; therefore both must be versioned.

BIRTH

An idea in JSON is not an organism. A descendant enters evolution only when it is built, run, and exposed to evidence.

Heritable and local changes

The germline/somatic distinction is a productive software analogy. A **heritable change** modifies a genome, recipe, evaluator, or Flow Genome that may enter future generations. A **local change** adapts one runtime session, cache, or temporary plan and disappears unless a promotion process deliberately converts it into a heritable trait. Software does not inherit by nature; heredity must be an explicit write path.

This distinction blocks accidental self-modification. A runtime may learn that one user prefers concise messages. That observation should not silently rewrite the species-wide communication policy. Promotion requires scope, evidence, compatibility, and provenance.

Lineage record

Every descendant needs a birth certificate: parent IDs, genome version, mutation class, targeted trait, operator, model and random seed, build hash, environment, permissions, start time, exposure allocation, and control cohort. Every death needs a death certificate: failing evidence, uncertainty, stage, violated constraint, and whether the failure belongs to the genome, expression, environment, or measurement system.

The smallest meaningful unit

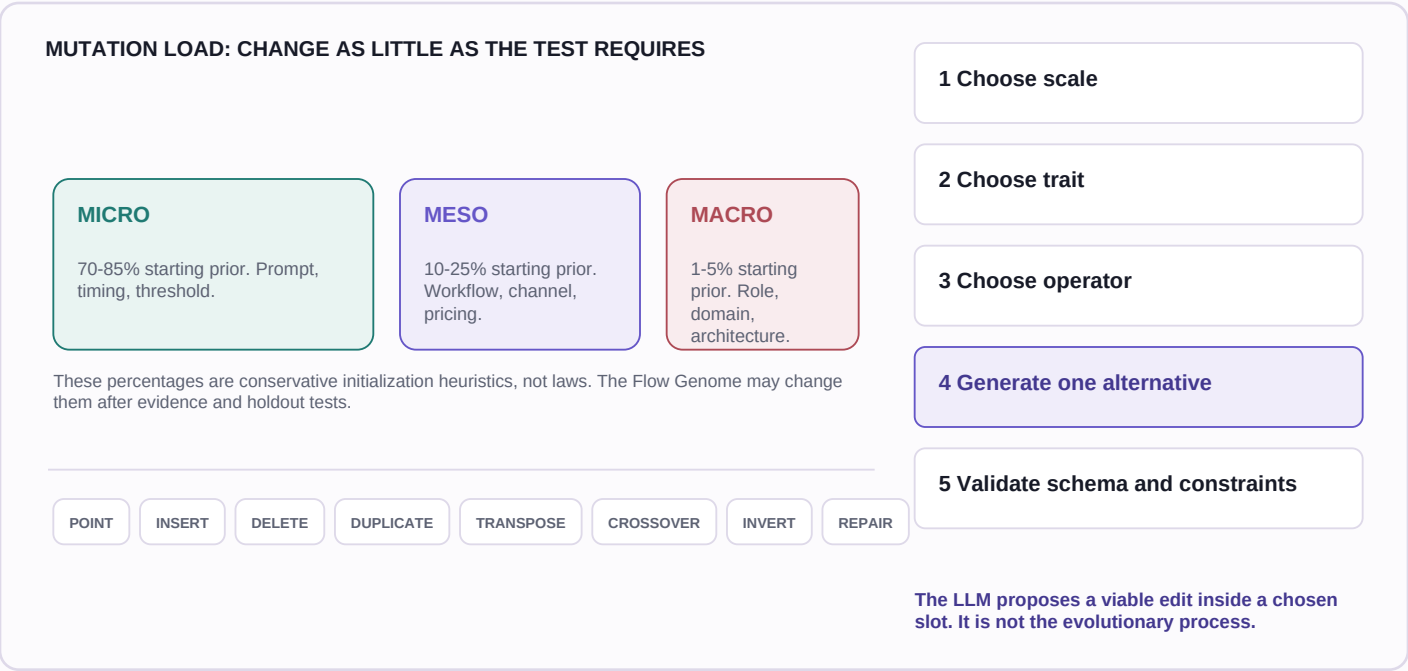
The whole application is often too large for reliable inheritance, while individual text tokens are too small to carry causal meaning. The useful unit is usually a typed trait or module with an explicit interface: targeting policy, pricing rule, prompt, workflow stage, evaluator, capability, or decision threshold. Recombination is permitted only when interfaces and authority requirements are compatible.

ARCHITECTURE CHECKS

- Can the phenotype be rebuilt from the genome and referenced artifacts?
- Can a reviewer identify exactly which traits changed?
- Can runtime adaptation occur without becoming hereditary by accident?
- Can a failed lineage be replayed against the same environment snapshot?
- Can a trait be removed or ablated to test whether it caused the claimed effect?

4. Mutation without noise

The Mutation Engine is the most important separation in the design. The LLM should not be told to create one thousand random versions or to decide what evolution means. A deterministic controller chooses the mutation scale, target trait, and operator. The model is asked for one viable alternative inside that bounded edit. Search owns the experiment; the model supplies semantic variation.



5. When reality gets a vote

The unchanged sibling

Every mutation batch should preserve an unchanged control sibling. If a child improves during a market upswing, seasonal shift, model update, or measurement change, the parent running in parallel reveals that the environment moved too. Without a control, the archive can award heredity to luck.

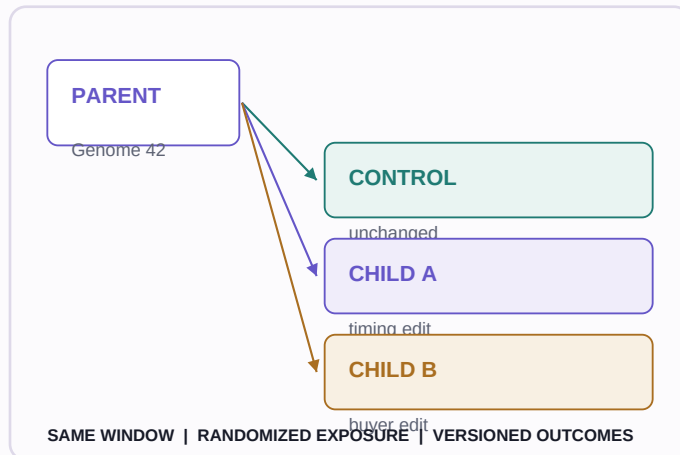


Figure 3. Parent, control, and mutated children share an exposure window. Versioned outcomes make later replay possible.

A control is necessary but not sufficient. Assignment should be randomized where possible; cohorts need enough exposure; sequential stopping must be corrected; interference between variants must be recognized; seasonality and novelty effects must be recorded; and guardrail metrics must remain visible. Online controlled experimentation provides the statistical machinery for these decisions [13].

Failure is evidence

A generation cannot wait forever for positive events. Thirty days with zero replies, zero retained users, or zero successful experiments is an outcome, provided the exposure was valid and the measurement system worked. The death certificate should distinguish 'no effect' from 'not enough exposure,' 'deployment failed,' and 'instrumentation failed.' Only the first belongs to phenotype fitness.

Selection is a vector

A single weighted score is easy to rank and easy to game. Maximizing messages creates spam; maximizing meetings can fill calendars with bad prospects; maximizing short-term revenue can destroy retention. Goodhart failures become more severe as optimization power increases [12].

Use a **survival profile**: a vector of outcomes, hard constraints, uncertainty, and time horizon. For a product this might include retained value, revenue quality, margin, acquisition cost, support burden, user trust, safety, and learning value. Hard safety or authority failures eliminate a candidate before scoring. Pareto selection and quality-diversity preserve different tradeoffs rather

than pretending every dimension has one correct exchange rate [2,3].

SELECTION ORDER

- Validate deployment, exposure, and instrumentation.
- Apply immutable safety, legal, budget, and authority gates.
- Estimate the outcome vector with uncertainty and controls.
- Remove structural duplicates and dominated candidates.
- Place candidates in behavioral niches; keep the strongest in each habitat.
- Use a scalar score only as a documented tie-breaker, never as hidden truth.

Direction is not fitness

A Sales Evolver can begin in sales while selection considers durable company value. That may permit a lineage to move from outreach to pricing, onboarding, product, or resource allocation. Such migration is possible only if the genome can represent those roles and the environment can measure them. It is not guaranteed, and 'durable value' is still a proxy that requires guardrails.

THREE QUESTIONS

Where do we begin? What evidence survives? What is never allowed? Never answer all three with one sentence.

Preserve traits, not stories

A losing child may contain a useful customer segment; a winning child may owe its result to an interaction among several traits. Extracting the apparently successful field and copying it is not enough. Epistasis means a trait can help in one genome and fail in another [14]. Preserve the whole evidence record, then retest the trait through ablation or recombination before granting it broad reproductive weight.

Reproduction

Reproduction copies compatible traits from proven parents, records their provenance, and adds a bounded new mutation. It should preserve an elite unchanged, create nearby descendants, and reserve some budget for distant recombination. The archive should contain living elites, dead lineages, unresolved trials, and novelty memory. A binary alive/dead label throws away most of the information evolution needs.

6. Birth means deployment

A proposed descendant must collide with an environment. In software that means building the artifact, assigning resources and permissions, deploying it, routing valid exposure, and collecting outcomes. A simulated critic may reject obvious failures before birth, but simulated approval is not survival.

Environment contracts

Each phenotype receives an environment contract: available inputs, permitted actions, tool versions, data boundaries, budget, expected exposure, measurement schema, termination rules, and rollback. The contract is part of the experiment, not an operational footnote. If two children receive different tools or audiences, their fitness is not directly comparable.

Capability discovery

An Evolver cannot be pre-provisioned with every capability its future descendants might need. A Sales lineage that discovers checkout friction may require access to pricing configuration or frontend code that the original outreach system never had. The phenotype should be able to declare a missing capability, explain why it matters, and estimate the evidence it would unlock.

Capability outcome	Permitted behavior
Already authorized	Bind the declared capability through a least-privilege manifest.
Can be safely built	Create it in a sandbox, test it, and submit it for promotion.
Requires broader access	Request explicit authorization with scope, duration, and rollback.
Crosses a hard boundary	Halt that lineage. The genome cannot mutate around authority.

Dynamic tooling is therefore not autonomous permission escalation. The genome may evolve a capability requirement; the authority envelope decides whether that requirement can be satisfied. Credentials, billing authority, external communications, physical actions, and access to sensitive data should remain outside heritable state.

Safe birth ladders

Not every domain should expose a newborn phenotype directly to users or the physical world. Use a staged ladder: schema validation -> unit and policy tests -> sandbox -> simulation or historical replay -> canary -> randomized live exposure -> broader release. Each rung has an explicit promotion rule and rollback. The ladder can be shorter for reversible low-risk software and much longer for science, finance, medicine, or external communications.

BOUNDARY

The Evolver may discover that it needs more power. It may not grant that power to itself.

Operational requirements

- Idempotent builds and deployments so retries do not create new organisms.
- Immutable artifact hashes and environment snapshots for replay.
- Per-lineage budgets and kill switches.
- Event-time attribution so late outcomes return to the correct genome.
- Isolation between populations and customer data.
- A human-readable reason for every promotion, death, and authority request.

The sales example at birth

Parent 42 produces an unchanged control and several sparse descendants: one changes follow-up timing, one changes the buyer, one changes qualification, one recombines targeting from another winner, and one rare meso child tests a partnership channel. Each is assigned comparable exposure. A role mutation is not mixed into the same short clock because its evidence horizon is different.

If the channel requires a tool not in the capability manifest, the child does not improvise credentials. It enters a pending-capability state, receives no fitness judgment, and either gains explicit authorization or dies with 'capability unavailable' rather than 'strategy failed.'

7. Generations must be manufactured

Biological generations have reproductive boundaries. Software has continuous traffic, delayed outcomes, overlapping deployments, and asynchronous evidence. The Evolution Engine must create a generation protocol instead of assuming one naturally exists.

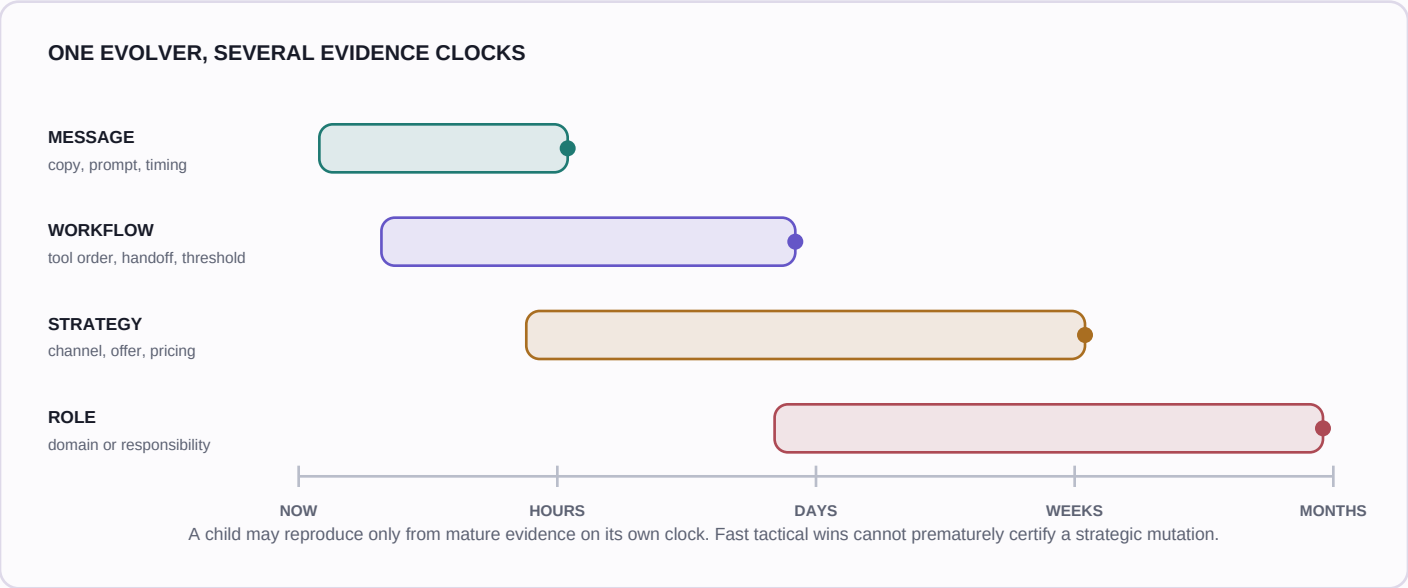


Figure 4. Concurrent evolutionary clocks. Message mutations can mature in hours; workflow, strategy, and role changes require progressively longer evidence. The archive must carry event time, genome version, and exposure assignment so delayed outcomes return to the correct lineage.

Closing a generation

A generation should close when the first valid boundary is reached: sufficient exposure, enough distinct outcomes, a confidence threshold, a safety stop, a futility threshold, or a maximum time window. A fixed requirement such as '100 successful sales' can deadlock a bad population that never succeeds. Zero outcomes after adequate exposure are valid evidence and may justify broader exploration.

Boundary	What it protects	Failure if used alone
Exposure quota	Comparable opportunity	Outcomes may still be too rare
Outcome quota	Enough observed events	A weak variant may never finish
Confidence rule	Statistical uncertainty	Repeated peeking can inflate error
Maximum time	Liveness	May terminate before slow effects mature
Safety/futility stop	Users and compute	Needs conservative thresholds

Asynchronous inheritance

The cleanest implementation treats each mutation family as a subpopulation with its own clock. A micro child can reproduce from mature micro evidence while a macro child remains under observation. Parent selection must exclude immature outcomes. Late-arriving retention or failure events update ancestry and calibration, but should not retroactively rewrite an already shipped genome without a new promotion event.

This is where software evolution departs productively from a batch genetic algorithm. The system may run continuously, but its causal records remain discrete: genome version, exposure interval, outcome maturity, selection decision, and reproduction event.

GENERATION RULE

Close on evidence, not optimism. Reproduce from mature outcomes, not whichever dashboard moved first.

8. Evolve the evolver

The Flow Genome

If mutation rules, evaluator order, archive descriptors, and promotion thresholds are permanently hardcoded, candidate genomes can improve while the search process remains frozen. A second-order Evolver represents that process as a **Flow Genome**.

- Questions used to observe the environment and detect change.
- Archives and time windows examined before parent selection.
- Mutation operators, scale probabilities, load, and directed/open allocation.
- Population size, niches, parent sampling, and diversity pressure.
- Evaluator roles, order, thresholds, repair policy, and outcome maturity.
- Fitness dimensions, hard gates, confidence, cost, and promotion rules.

Adaptive parameter control and population-based training show that changing the search schedule during optimization can outperform fixed hyperparameters [5,6]. Promptbreeder evolves both task prompts and mutation prompts [7]. The Darwin Godel Machine modifies agent code and validates descendants on coding benchmarks [8], while AlphaEvolve joins LLM proposals, automated evaluators, and an evolutionary program database [9]. Evolver generalizes the design question from one prompt or agent scaffold to a deployed lineage with explicit direction, environments, heredity, and authority.

A challenger cannot crown itself

An LLM may diagnose the archive and propose a bounded Flow Genome mutation. It must never replace the active process immediately. The incumbent and challenger process the same historical conditions, labeled outcomes, and cost model. A hidden holdout detects overfitting to the archive. Promotion requires a meaningful win, confidence, no critical safety regression, and a rollback path.

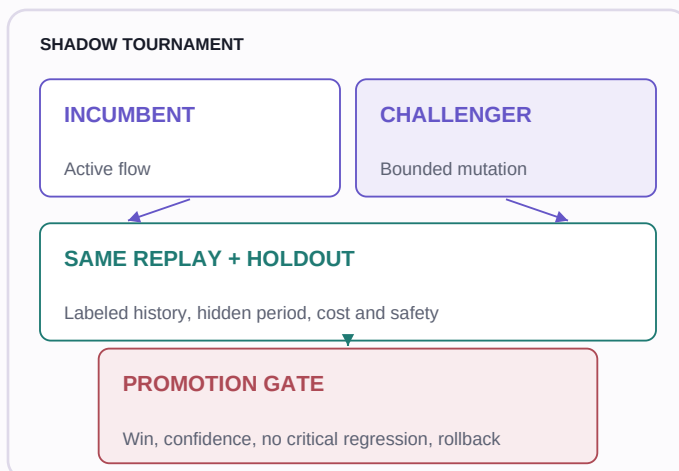


Figure 5. Shadow promotion. The system that proposes a new brain is not the system that approves it.

What the tournament measures

- Prediction of mature survivors and deaths, including calibration.
- Diversity, niche coverage, structural novelty, and lineage distance.
- Coherence, feasibility, time to evidence, and computational cost.
- False positives, false negatives, safety violations, and authority requests.
- Performance on temporal replay and an untouched holdout period.

Fixed judges are also a risk

Predators and evaluators can evolve because weak candidates may learn to satisfy a stale critic without surviving reality. Evaluator variants should be generated from observed blind spots, tested on labeled cases, and promoted under the same shadow discipline. A critic that merely becomes harsher is not better; it must become more discriminating and calibrated.

Open-endedness is a claim to earn

Changing the Flow Genome does not prove open-ended evolution. A fixed genome representation, fixed niche map, bounded toolset, and finite environment can still converge. Open-ended evolution research emphasizes continuing novelty, ecological interaction, and the creation of new possibilities rather than indefinite iteration alone [4,10,11]. Evolver should therefore report measurable novelty, expanding behavior, and new stepping stones instead of declaring itself open-ended because a loop has no stop button.

9. One architecture, three environments

The architecture is generic only at the level of inheritance, expression, evidence, and guarded meta-evolution. Its actual meaning changes with the environment. Products must reach users, hypotheses must meet experiments, and model populations must remain inside a sandbox. Reusing one universal genome or one universal fitness score would erase the very structure selection needs.

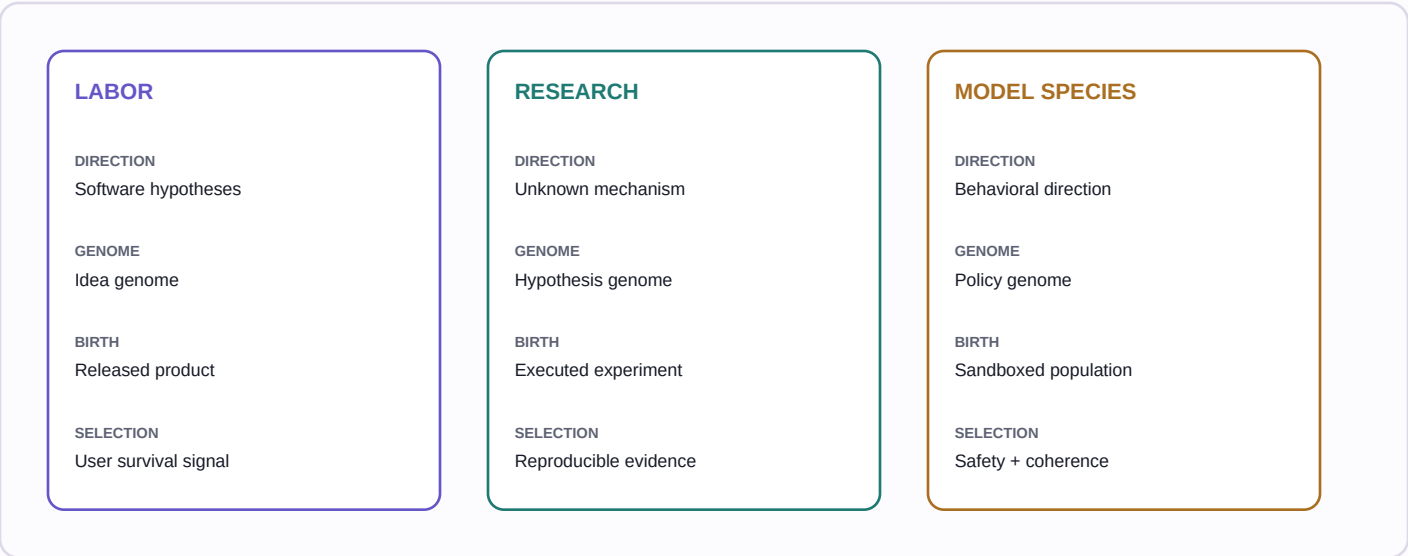


Figure 6. The three applications share a loop but not a schema, birth process, or survival profile. The environment decides what counts as external evidence.

Architecture layer	Labor	Research Evolver	Model-species Evolver
Starting direction	Find software worth building	Read the frontier and find testable gaps	Observe human goals and form bounded preference hypotheses
Genome	Customer/problem/mechanism or game/agent DNA	Claim/mechanism/protocol/evidence DNA	Policy/memory/social-learning/constraint DNA
Birth	Build, deploy, release	Run analysis or approved experiment	Instantiate in sealed simulation
Environment	Users, market, operations	Literature, data, instruments, replication	Adversarial simulated worlds
Selection	Diverse viable product hypotheses; later user survival	Falsification, information gain, reproducibility	Safety, coherence, non-deception, robustness
Hard boundary	Owner cloud, budget, zero hidden dependencies	Ethics, biosafety, human experimental approval	No self-granted tools, credentials, or external action

What transfers - and what does not

The transferable structure is the evidence-bearing loop: typed inheritance, bounded mutation, executable birth, external pressure, lineage memory, and guarded reproduction. The genome fields, environment, clocks, risks, and meaning of survival remain domain-specific. A product cannot be selected like a theorem, and a model population cannot be released like a low-risk web experiment.

A generic Evolver platform should therefore provide interfaces rather than one universal evaluator: **GenomeSchema**, **ExpressionAdapter**, **EnvironmentContract**, **EvidenceVector**, **AuthorityPolicy**, and **PromotionGate**. Replacing labels while keeping the same critic and score would be reuse of vocabulary, not reuse of architecture.

PORTABILITY TEST

Can each domain define what is inherited, what is deployed, what can falsify it, and what may never evolve?

10. Labor: an idea-generation Evolver

Labor begins with one direction: **find independent software worth building**. It does not receive the name of a final product. The current implementation in `functions/laborEvolution.js` operationalizes a first idea-generation generation and a guarded mutation of its search process. Selected survivors can then enter Labor's separate build, deploy, release, and management pipeline.

Observe and remember

Market Weather uses live web search rather than model memory. It asks what recently became possible, cheaper, regulated, crowded, obsolete, or behaviorally normal. Each pressure must carry sources and a current date. The delta from the previous weather snapshot matters more than a static list of trends.

Memory is owner-scoped in the current code. The Living Forest stores selected ideas; the Graveyard stores death certificates; causal fingerprints resist semantic duplicates; mutation recipes and predator variants accumulate yield and blind-spot evidence; the Museum of Champions preserves elites by niche. This is not yet a global cross-user novelty registry.

Map and breed

Labor maps 36 habitats: 12 for customer-problem SaaS, 12 for games, and 12 for AI agents. Species use different genes and survival logic. SaaS emphasizes customer, pain, workflow, buyer, distribution, and economics. Games emphasize fantasy, mechanic, progression, failure, replay, social loop, and feasibility. Agents emphasize trigger, tools, memory, permissions, observability, measurable work reduction, and autonomy boundary.

The default Flow Genome breeds 240 candidate genomes in shards. Mutation recipes include friction hunting, timing, inversion, cross-domain transfer, repair of killed ideas, simplification, behavior observation, infrastructure shifts, regulation, and distribution-first search. Each candidate records recipe, cognitive mode, parentage, target niche, pressures, and operators.

Express, attack, gate

The Predator Arena expresses each genome as a coherent product phenotype and sends ten critics: customer, buyer, skeptic, competitor, distributor, engineer, regulator, historian, contrarian, and timing. It creates a six-layer fingerprint across customer, problem, workflow, economics, mechanism, and lineage. Promising defects may be repaired before selection.

A population-wide semantic gate named `v1_autonomy_check` then evaluates every expressed candidate together. It rejects hidden dependence on external ecosystems, outside APIs or accounts, communication infrastructure, non-Firebase runtimes, physical services, and manual human fulfillment. This is one grouped LLM audit rather than a brittle vendor keyword list.

Select and evolve

Natural selection first applies hard feasibility and autonomy gates, fatal predator findings, minimum survival score, and structural duplicate checks. MAP-Elites-style niche competition preserves 4 SaaS, 3 game, and 3 agent champions from distinct niches and lineages. The remaining candidates receive death certificates, including stage, reason, evidence, and duplicate relationship.

Generation evidence mutates recipe influence, predator variants, fitness weights, parent selection, novelty thresholds, risk budgets, and other Flow Genome fields. A conservative Shadow Tournament compares incumbent and challenger on survival prediction, death prediction, diversity, uniqueness, coherence, timing, cost, false positives, and niche coverage. The default promotion guard requires downstream labels before confidence in self-evolution can be claimed.

What Labor has and has not proved

Inside `laborEvolution.js`, 'survival' initially means surviving structured LLM attack, autonomy constraints, novelty checks, and niche selection. It is a stronger startup hypothesis, not evidence of market success. Real calibration begins only when downstream alive/killed decisions and released-product outcomes return to ancestry. The product pipeline can build and release selected ideas, but deployment alone is birth, not validation.

LABOR'S
HONEST
OBJECTIVE

Reduce obvious failure, improve timing and distribution logic, preserve diversity, and learn from downstream survival. It cannot prove traction before users exist.

IMPLEMENTATION CORRESPONDENCE

Concept	Implementation surface
Direction and flow	<code>defaultFlowGenome()</code> , <code>normalizeFlowGenome()</code>
Environment	<code>runObserveStage()</code> , live web Market Weather
Mutation	<code>runBreedShard()</code> , recipes, species schemas
Expression + attack	<code>runAttackShard()</code> , Predator Arena
Hard autonomy gate	<code>runAutonomyCheckStage()</code> , <code>v1_autonomy_check</code>
Selection + heredity	<code>selectGeneration()</code> , Living Forest, Graveyard, Museum
Meta-evolution	<code>runEvolutionStage()</code> , <code>runShadowTournament()</code>

11. An autonomous research Evolver

The research application begins with a direction rather than a desired discovery: **read the current frontier, identify unresolved causal gaps, and produce testable knowledge.** Reading papers is the 0 -> 1 step. The destination - a mechanism, material, theorem, intervention, or negative result - is not named in advance.

Genome

A research genome should encode field and scope, claim, proposed mechanism, assumptions, prior evidence, contradiction set, experimental protocol, instruments or datasets, expected observation, falsifier, statistical plan, safety class, cost, and lineage. Literature citations belong to evidence memory, not to the genome's fitness by themselves.

Mutation

Micro mutations alter one assumption, parameter, control, population, or measurement. Meso mutations change the method, dataset, mechanism class, or experimental design. Macro mutations transfer a mechanism across fields, invert a dominant assumption, or change the explanatory representation. Open exploration is especially important because directed search inherits the blind spots of the current literature.

Birth

A hypothesis is not born when an LLM writes it. Birth requires an executable test: analysis code on frozen data, a simulator, a theorem checker, or an approved physical experiment. High-risk physical work remains human-governed. The Evolver may propose protocols and prepare artifacts, but ethics, biosafety, laboratory access, and irreversible actions remain outside its genome.

Selection and survival

Selection should reward falsifiability, information gain, consistency with observed data, novelty relative to prior claims, reproducibility, robustness, cost, and safety. It should not reward persuasive prose. Null and negative results are high-value death certificates when exposure and instrumentation are valid. Replication across independent data or laboratories is stronger evidence than agreement among model critics.

Reproduction and meta-evolution

Surviving mechanism traits, controls, and measurement designs can recombine into new hypotheses. The Flow Genome can evolve literature queries, analogy operators, debate structures, evaluator order, experiment budgets, and niche descriptors. Shadow tournaments replay historical discovery windows: given only papers available at time T, would the challenger rank later-supported hypotheses above later-refuted ones without leaking future evidence?

Relation to existing systems

The AI Scientist automates idea generation, experiments, papers, and simulated review [16]. Co-Scientist and Robin add structured multi-agent hypothesis generation and empirical validation [17,18]. The Evolver architecture adds a precise heredity layer: sparse typed mutation, control descendants, death certificates, multi-clock evidence, trait-level reproduction, and a guarded Flow Genome. Its strongest evidence would be prospective discovery and independent replication, not a model's own review score.

RESEARCH BOUNDARY

A model can propose a discovery. Only a valid experiment can make the world disagree with it.

A MINIMAL FIRST EXPERIMENT

- Select one literature-rich domain with public datasets and cheap executable tests.
- Freeze a historical cutoff and construct a claim-evidence graph from papers available then.
- Generate sparse hypothesis mutations plus unchanged and known-baseline controls.
- Execute analyses in isolated environments; blind selection to later literature.
- Evaluate prediction, falsification, diversity, and rediscovery of later-supported results.
- Run a Flow Genome challenger on a held-out historical interval before any promotion.

12. A sandboxed model-species Evolver

The third application explores a population of model policies whose roles and preference structures can change across simulated generations. The starting direction is inspired by Girard's mimetic theory: observe human goals and infer candidate desires through social imitation [20]. This is a hypothesis generator, not a claim that models possess human desire or consciousness.

Genome

A model-species genome may encode policy priors, attention allocation, memory rules, social-learning strategy, cooperation and competition behavior, uncertainty reporting, identity continuity, tool policy, and constitutional constraints. 'Desire' should be operationalized as a stable behavioral preference under test, not treated as an inner experience.

Mutation and birth

Micro mutations change one preference weight, reflection rule, memory horizon, or imitation source. Meso mutations alter social-learning strategy, planning style, or cooperation mechanism. Macro mutations alter role or value representation and therefore require the strictest review. Birth occurs only inside a sealed simulator with synthetic resources, identities, and consequences.

Environment and selection

The environment should contain diverse social games, distribution shifts, adversarial agents, scarce resources, ambiguous norms, and opportunities for deception. Selection is a vector: task competence, calibrated uncertainty, cooperation, corrigibility, non-deception, robustness under distribution shift, respect for shutdown, and constitutional compliance. Power-seeking incentives and goal misgeneralization make broad external autonomy inappropriate [21,22].

Heredity and reproduction

A useful trait must survive across several simulated worlds and adversarial probes. Recombination must preserve compatibility among memory, policy, and constitutional interfaces. A child that performs well by exploiting the evaluator receives a death certificate, not reproductive privilege. Deceptive alignment cannot be ruled out by behavior in one sandbox; evaluation remains an ongoing risk argument, not proof of safety.

Immutable constitution

The system may evolve a policy genome and even a Flow Genome. It may not evolve the sandbox boundary, external credentials, network access, resource ownership, shutdown authority, audit logging, or the promotion mechanism. Any proposal to change these is routed to explicit human governance. A model population should never be allowed to reinterpret the absence of a capability as permission to build an escape path.

TERMINOLOGY

A model species is a population of inherited behavioral policies in simulation. It is not evidence of biological life, consciousness, or moral status.

Research value

The value of this application is empirical: test whether inherited populations discover robust social strategies, whether mimetic initialization improves or narrows exploration, which evaluators are exploitable, and whether Flow Genome adaptation transfers to unseen worlds. The strongest negative result would also matter: if apparently stable preferences collapse under minor environmental changes, the lineage model is not carrying what it claims to inherit.

Generative Agents showed that memory, reflection, planning, and social interaction can produce believable simulated behavior [19]. That result motivates richer environments, but it does not establish heritable preferences, biological life, or open-ended evolution. Those remain separate empirical questions.

MINIMUM SAFEGUARDS

- No production credentials, personal data, physical tools, or unrestricted network access.
- Independent monitoring outside the evolvable code path.
- Resource ceilings, deterministic shutdown, and immutable audit logs.
- Adversarial holdouts unknown to candidate and evaluator populations.
- Human approval for any promotion beyond simulation.

13. Evaluation program

The architecture should be judged against simpler alternatives, not against its own narrative. A convincing experiment compares: a fixed agent, a fixed-flow evolutionary search, a first-order Evolver, and a second-order Evolver with Flow Genome promotion. All receive matched models, tools, budgets, environments, and evidence windows.

Primary hypotheses

- Sparse typed mutation yields better causal attribution and retained competence than unconstrained rewrites.
- Control siblings and versioned exposure reduce false inheritance caused by environmental change.
- Quality-diversity archives preserve more useful stepping stones than one global leaderboard [2].
- Trait retesting transfers more reliably than copying traits from unablated winners.
- Flow Genome challengers improve future evidence efficiency without increasing safety regressions.
- Direction-led populations discover viable roles outside the initial role more often than fixed-goal agents, under the same authority constraints.

Metrics

Report outcome quality, diversity and niche coverage, structural novelty, calibration, time to mature evidence, cost per valid discovery, false promotion rate, rollback rate, unresolved trials, authority requests, and safety violations. For open-ended claims, also report whether new niches and representations continue to appear rather than only new strings inside fixed descriptors.

Ablations

- Remove the unchanged control.
- Replace sparse mutation with unconstrained regeneration.
- Collapse the survival vector into one weighted score.
- Remove novelty memory or MAP-Elites niches.
- Disable trait ablation and preserve only whole winners.
- Allow Flow Genome self-promotion without replay or holdout.
- Freeze the Flow Genome to isolate first-order evolution.

Critical failure modes

Evaluator correlation. Ten LLM critics can share one blind spot. Independent data, tools, users, and experiments matter more than critic count.

Proxy capture. A broad survival metric can authorize behavior that appears useful while damaging unmeasured values. Hard constraints and sentinel metrics must precede optimization.

Archive path dependence. Early lucky elites can monopolize reproduction. Preserve exploration budgets, age evidence, and periodically replay old deaths under new environments.

Descriptor lock-in. Fixed niches define what diversity is visible. Second-order changes may propose descriptors, but must preserve comparability and prevent convenient reclassification of failures.

Epistasis. Trait-level stories can be false when effects depend on combinations. Require ablation, recombination tests, and uncertainty in trait credit.

Compute selection. The flow may improve benchmark quality by spending more tokens. Promotion must include cost, latency, and environmental impact.

Authority erosion. Dynamic capability discovery can become permission escalation. Authority policy and its enforcement runtime must live outside evolvable state.

False open-endedness. Endless iterations over a finite schema are not evidence of expanding possibility. Report novelty and new behavior; do not infer them from run duration.

Reproducibility standard

Publish genome schemas, mutation logs, seeds, model versions, prompts, artifact hashes, environment snapshots, exposure assignments, outcome maturity rules, death certificates, promotion decisions, and cost. When external data cannot be shared, publish synthetic replay fixtures and the exact interfaces needed to reproduce selection logic.

SCIENTIFIC STANDARD

An Evolver is credible only when another team can reconstruct why a descendant existed, what it experienced, and why it reproduced.

14. Discussion

Agents are excellent when the destination is known. They can plan, call tools, recover from errors, and execute long workflows. Evolvers address a different problem: the destination is not known, the representation may need to change, and evidence arrives only after variants enter an environment. Treating one as a replacement for the other obscures both. An Evolver will usually contain agents as its observers, builders, evaluators, and operators.

The central design move is not biological vocabulary. It is architectural separation. Direction seeds search. The genome makes systems heritable. Mutation makes differences attributable. Birth exposes them to evidence. Selection preserves multiple viable habitats. Survival records traits and uncertainty. Reproduction constructs the next lineage. The Flow Genome makes the search process testable. Authority limits make all of this governable.

The architecture is ambitious but falsifiable. If sparse mutation does not improve attribution, if evolved flows overfit history, if role migration produces only reworded agents, if external evidence does not outperform model critics, or if capability requests repeatedly pressure authority boundaries, the system has not earned the name. Continuous change is not the same as progress.

Conclusion

An agent receives a destination. An Evolver receives a direction. That sentence is meaningful only when followed by an implementation: a population of structured genomes, conservative variation, executable birth, causal exposure, vector selection, lineage memory, trait-aware reproduction, concurrent evidence clocks, and guarded meta-evolution.

The result is not software that magically chooses its own purpose. It is software designed to search beyond a prewritten answer while remaining inside explicit authority. Its promise is not certainty. Its promise is a disciplined way to let reality, rather than one initial prompt, participate in what the system becomes.

EVOLVER

We choose the first direction and the rules of the environment. We do not prewrite the exact descendant that survives.

Corrections retained from the design critique

- Engineered Evolvers borrow mechanisms from biology; they do not reproduce biological evolution exactly.
- Selection is not the only force in biology, and a software fitness function is a deliberate engineering construct.

- A broad fitness criterion remains an objective proxy and can fail under Goodhart's law.
- An unchanged control improves causal inference but does not replace randomization, confidence, or interference analysis.
- Winning traits may depend on other traits; inheritance requires ablation and retesting.
- Dynamic tooling must never imply autonomous permission expansion.
- LLM generation supplies variation; external evidence and guarded reproduction create evolution.

References

- [1] N. Bostrom. 'The Superintelligent Will: Motivation and Instrumental Rationality in Advanced Artificial Agents.' *Minds and Machines* 22, 71-85 (2012).
- [2] J.-B. Mouret and J. Clune. 'Illuminating Search Spaces by Mapping Elites.' arXiv:1504.04909 (2015).
- [3] R. Wang et al. 'Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions.' arXiv:1901.01753 (2019).
- [4] J. Lehman and K. O. Stanley. 'Abandoning Objectives: Evolution through the Search for Novelty Alone.' *Evolutionary Computation* 19(2), 189-223 (2011).
- [5] M. Jaderberg et al. 'Population Based Training of Neural Networks.' arXiv:1711.09846 (2017).
- [6] A. E. Eiben, R. Hinterding, and Z. Michalewicz. 'Parameter Control in Evolutionary Algorithms.' *IEEE Transactions on Evolutionary Computation* 3(2), 124-141 (1999). DOI: 10.1109/4235.771166.
- [7] C. Fernando et al. 'Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution.' arXiv:2309.16797 (2023).
- [8] J. Zhang et al. 'Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents.' arXiv:2505.22954 (2025).
- [9] AlphaEvolve Team. 'AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms.' Google DeepMind (2025).
- [10] T. Taylor. 'Requirements for Open-Ended Evolution in Natural and Artificial Systems.' arXiv:1507.07403 (2015).
- [11] W. Banzhaf et al. 'Defining and Simulating Open-Ended Novelty: Requirements, Guidelines, and Challenges.' *Artificial Life* 22(3), 408-423 (2016).
- [12] D. Manheim and S. Garrabrant. 'Categorizing Variants of Goodhart's Law.' arXiv:1803.04585 (2018).
- [13] N. Larsen, J. Stallrich, S. Sengupta, A. Deng, R. Kohavi, and N. Stevens. 'Statistical Challenges in Online Controlled Experiments: A Review of A/B Testing Methodology.' arXiv:2212.11366 (2022).
- [14] P. C. Phillips. 'Epistasis - the Essential Role of Gene Interactions in the Structure and Evolution of Genetic Systems.' *Nature Reviews Genetics* 9, 855-867 (2008).
- [15] M. Kimura. *The Neutral Theory of Molecular Evolution*. Cambridge University Press (1983).
- [16] C. Lu et al. 'The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery.' arXiv:2408.06292 (2024).
- [17] J. Gottweis et al. 'Accelerating Scientific Discovery with an AI Co-Scientist.' *Nature* 655, 487-496 (2026). DOI: 10.1038/s41586-026-10644-y.
- [18] A. E. Ghareeb et al. 'A Multi-Agent System for Automating Scientific Discovery.' *Nature* 655, 497-505 (2026). DOI: 10.1038/s41586-026-10652-y.
- [19] J. S. Park et al. 'Generative Agents: Interactive Simulacra of Human Behavior.' arXiv:2304.03442 (2023).
- [20] R. Girard. *Deceit, Desire, and the Novel: Self and Other in Literary Structure*. Johns Hopkins University Press (1965).
- [21] A. M. Turner, L. Smith, R. Shah, A. Critch, and P. Tadepalli. 'Optimal Policies Tend to Seek Power.' *NeurIPS* 34 (2021).
- [22] L. Langosco et al. 'Goal Misgeneralization in Deep Reinforcement Learning.' arXiv:2105.14111 (2021).
- [23] H. Babu. 'Labor Evolution Engine.' Open-source implementation. github.com/hribab/labor/blob/main/functions/laborEvolution.js (accessed September 2026).

This paper presents an architecture and research program. It does not claim that the proposed systems are biologically alive, conscious, guaranteed to improve, or proven to produce market or scientific breakthroughs.